DevOps Assignment II

Carrier Selection Agent

AI-Powered Logistics

Implementing DevOps Practices Throughout the Development Lifecycle

1. Description of the Application Developed

Application Name: Carrier Selection Agent – AI-Powered Logistics

Overview

The Carrier Selection Agent is an AI-powered logistics web application that helps users automatically select the best shipping carrier based on shipment details. The backend is built with FastAPI (Python) which exposes REST API endpoints, and the frontend is built with React which provides an interactive user interface.

Key Features

- React frontend with a shipment input form (weight, destination, budget, priority)
- FastAPI backend with an AI rule-based carrier scoring engine
- REST API returns the best carrier with cost and delivery time
- Fully containerized with Docker and Docker Compose
- Automated CI/CD pipeline using GitHub Actions

Tech Stack

Layer	Technology
Backend	Python 3.11 + FastAPI + Uvicorn
Frontend	React 18 + Axios
Containerization	Docker + Docker Compose
CI/CD	GitHub Actions
Version Control	Git + GitHub
Deployment	Docker Compose (Local)

Project Folder Structure

```
carrier-selection-agent/
├── backend/
│   ├── main.py
│   ├── carrier_agent.py
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/
│   ├── src/
│   │   ├── App.jsx
│   │   └── index.js
│   ├── package.json
│   └── Dockerfile
├── docker-compose.yml
└── .github/
    └── workflows/
        └── ci.yml
```

2. Use of Version Control (Git/GitHub)

Repository Initialization

```
# Initialize Git repository
git init carrier-selection-agent
cd carrier-selection-agent

# Create folder structure
mkdir -p backend frontend .github/workflows

# First commit
git add .
git commit -m "Initial commit: Project structure setup"

# Push to GitHub
git remote add origin https://github.com/yourusername/carrier-selection-agent.git
git push -u origin main
```

Branching Strategy

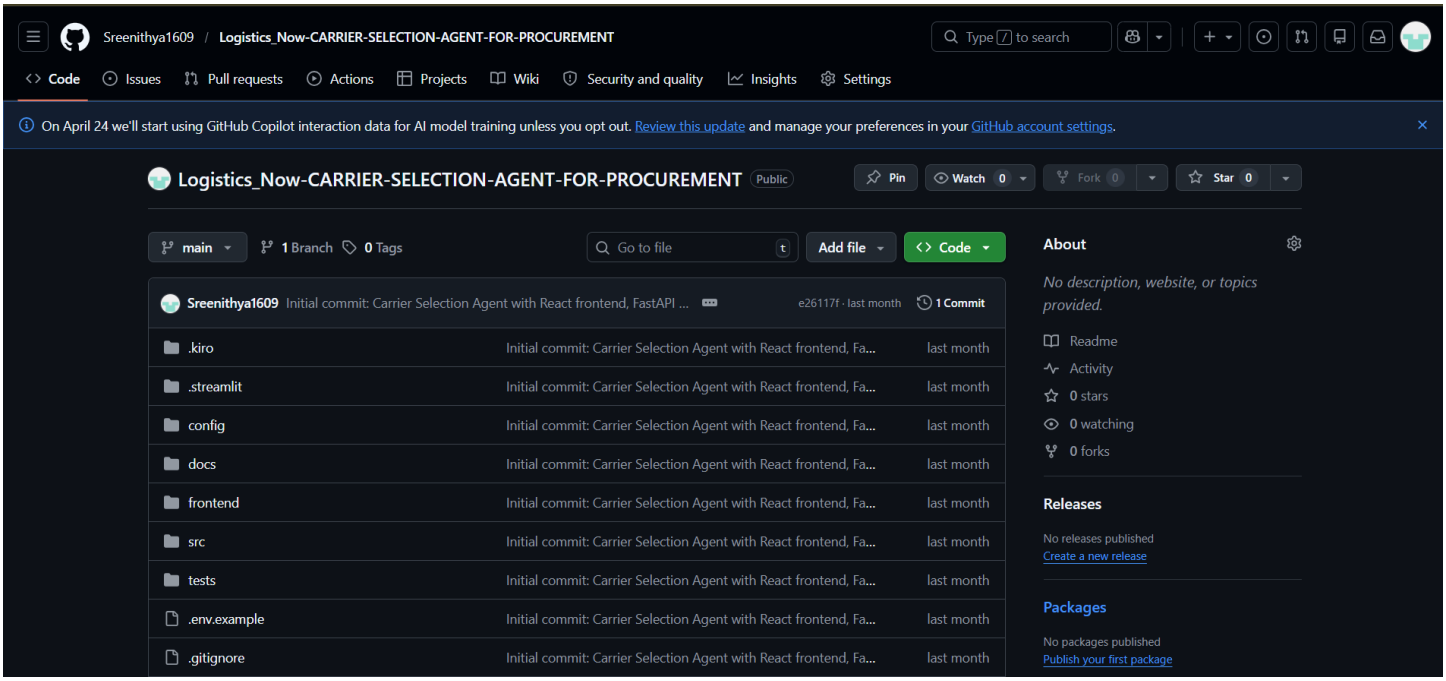
Branch	Purpose
main	Stable production-ready code
dev	Integration branch
feature/fastapi	Backend API development
feature/react-ui	React frontend development
feature/docker	Docker & Compose setup
feature/cicd	GitHub Actions workflow

Key Git Commands

```
# Create and switch to feature branch
git checkout -b feature/fastapi

# Stage and commit changes
git add backend/main.py
git commit -m "feat: Add FastAPI carrier selection endpoint"

# Push branch to GitHub
git push origin feature/fastapi

# Merge into main after review
git checkout main
git merge feature/fastapi

# Tag the release
git tag v1.0.0
git push origin v1.0.0
```

Sample .gitignore

```
__pycache__/
*.pyc
.env
venv/
node_modules/
build/
*.log
.DS_Store
```



3. Implementation of Continuous Integration (CI)

CI is implemented using GitHub Actions. The pipeline runs automatically on every push or pull request to the main branch.

File: .github/workflows/ci.yml

```
name: CI Pipeline - Carrier Selection Agent

on:
  push:
    branches: [ main, dev ]
  pull_request:
    branches: [ main ]

jobs:
  backend-ci:
    name: Backend - FastAPI Tests
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3
      - name: Set up Python 3.11
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'
      - name: Install Backend Dependencies
        run: |
          cd backend
```

```
          pip install -r requirements.txt
      - name: Run Backend Tests
        run: |
          cd backend
          pytest tests/ -v
      - name: Lint Backend Code
        run: |
          pip install flake8
          flake8 backend/ --max-line-length=100

  frontend-ci:
    name: Frontend - React Build
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3
      - name: Set up Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'
      - name: Install Frontend Dependencies
        run: |
          cd frontend
          npm install
      - name: Build React App
        run: |
          cd frontend
          npm run build

  docker-build:
    name: Docker Compose Build
    runs-on: ubuntu-latest
    needs: [backend-ci, frontend-ci]
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3
      - name: Build Docker Images
        run: docker-compose build
      - name: Notify Success
        run: echo 'All CI stages passed successfully!'
```

CI Stages

Stage	Tool	Purpose
1. Backend CI	pytest + flake8	Run tests and lint Python code
2. Frontend CI	Node.js + npm	Install deps and build React app
3. Docker Build	Docker Compose	Build both container images

4. Containerization Using Docker

Backend Code

backend/carrier_agent.py

```
CARRIERS = [
    {"name": "FedEx",     "base_cost": 15, "speed_days": 2, "max_weight": 70},
    {"name": "DHL",       "base_cost": 12, "speed_days": 3, "max_weight": 50},
    {"name": "UPS",       "base_cost": 10, "speed_days": 4, "max_weight": 100},
    {"name": "Blue Dart", "base_cost": 8,  "speed_days": 5, "max_weight": 30},
]

def select_carrier(weight: float, budget: float, priority: str):
    eligible = [
        c for c in CARRIERS
        if c['max_weight'] >= weight and c['base_cost'] <= budget
    ]
    if not eligible:
        return None
    if priority == 'speed':
        return min(eligible, key=lambda c: c['speed_days'])
    return min(eligible, key=lambda c: c['base_cost'])
```

backend/main.py

```
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from carrier_agent import select_carrier

app = FastAPI(title="Carrier Selection Agent API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

class ShipmentRequest(BaseModel):
    weight: float
    budget: float
    priority: str  # 'speed' or 'cost'

@app.get("/")
def root():
    return {"message": "Carrier Selection Agent API is running"}

@app.post("/select-carrier")
def select(req: ShipmentRequest):
    result = select_carrier(req.weight, req.budget, req.priority)
    if not result:
        raise HTTPException(status_code=404, detail="No suitable carrier found")
    return {"status": "success", "carrier": result}
```

backend/requirements.txt

```
fastapi==0.110.0
uvicorn==0.29.0
pydantic==2.6.0
pytest==7.4.0
httpx==0.27.0
flake8==6.1.0
```

backend/Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Frontend Code

frontend/src/App.jsx

```
import { useState } from "react";
import axios from "axios";

function App() {
  const [form, setForm] = useState({ weight: "", budget: "", priority: "cost" });
  const [result, setResult] = useState(null);
  const [error, setError] = useState("");

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError(""); setResult(null);
    try {
      const res = await axios.post("http://localhost:8000/select-carrier", {
        weight: parseFloat(form.weight),
        budget: parseFloat(form.budget),
        priority: form.priority,
      });
      setResult(res.data.carrier);
    } catch (err) {
      setError("No suitable carrier found. Try adjusting your inputs.");
    }
  };

  return (
    <div style={{ maxWidth: "500px", margin: "50px auto" }}>
      <h2>Carrier Selection Agent</h2>
      <form onSubmit={handleSubmit}>
        <input type='number' placeholder='Weight (kg)'
          onChange={(e) => setForm({ ...form, weight: e.target.value })} required />
        <input type='number' placeholder='Budget ($)'
          onChange={(e) => setForm({ ...form, budget: e.target.value })} required />
        <select onChange={(e) => setForm({ ...form, priority: e.target.value })}>
          <option value='cost'>Lowest Cost</option>
          <option value='speed'>Fastest Delivery</option>
        </select>
        <button type='submit'>Find Best Carrier</button>
      </form>
      {result && <div><h3>Recommended: {result.name}</h3>
        <p>Cost: ${result.base_cost} | Delivery: {result.speed_days} days</p></div>}
      {error && <p style={{ color: 'red' }}>{error}</p>}
    </div>
  );
}
export default App;
```

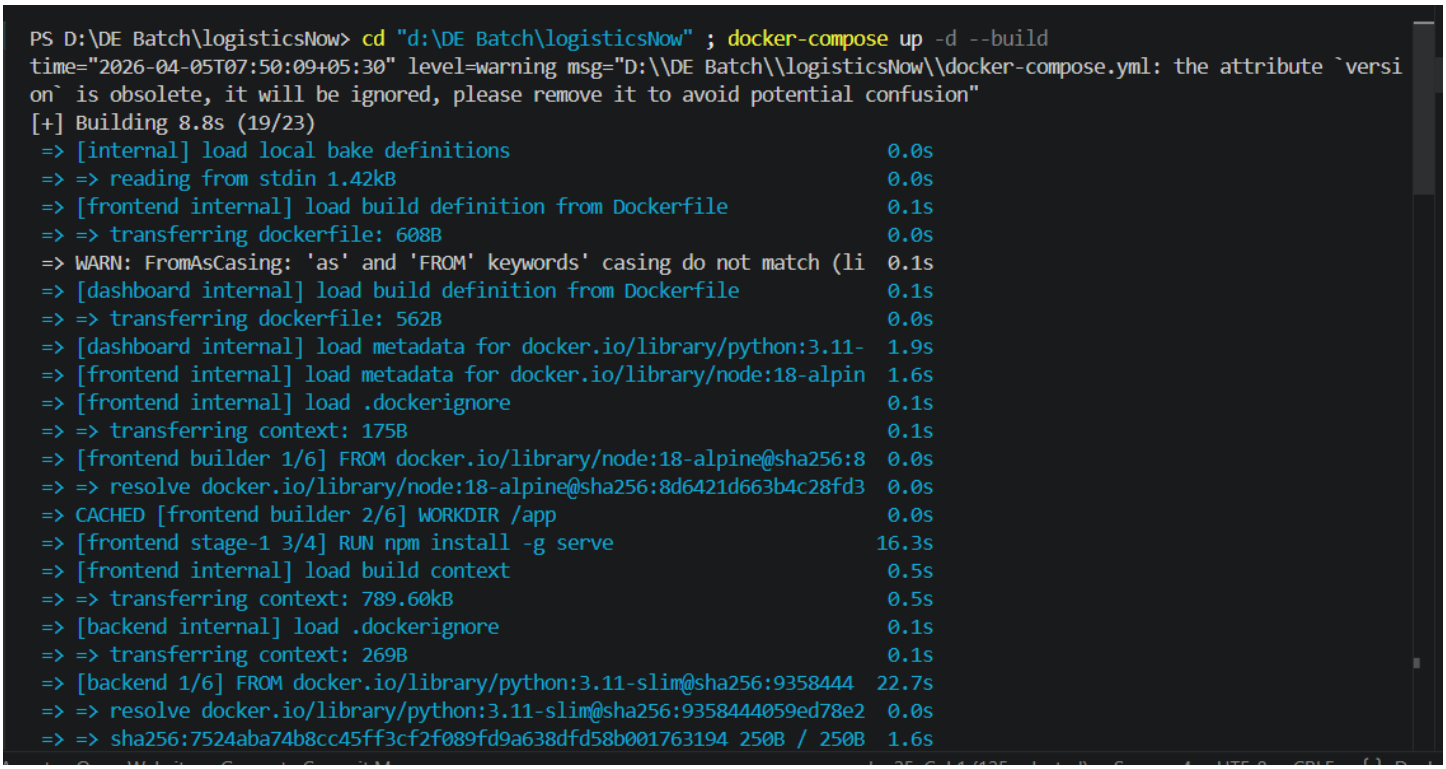
frontend/Dockerfile

```
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Docker Compose

docker-compose.yml

```
version: '3.8'
services:
  backend:
    build: ./backend
    container_name: carrier-backend
    ports:
      - '8000:8000'
    restart: unless-stopped

  frontend:
    build: ./frontend
    container_name: carrier-frontend
    ports:
      - '3000:80'
    depends_on:
      - backend
    restart: unless-stopped
```

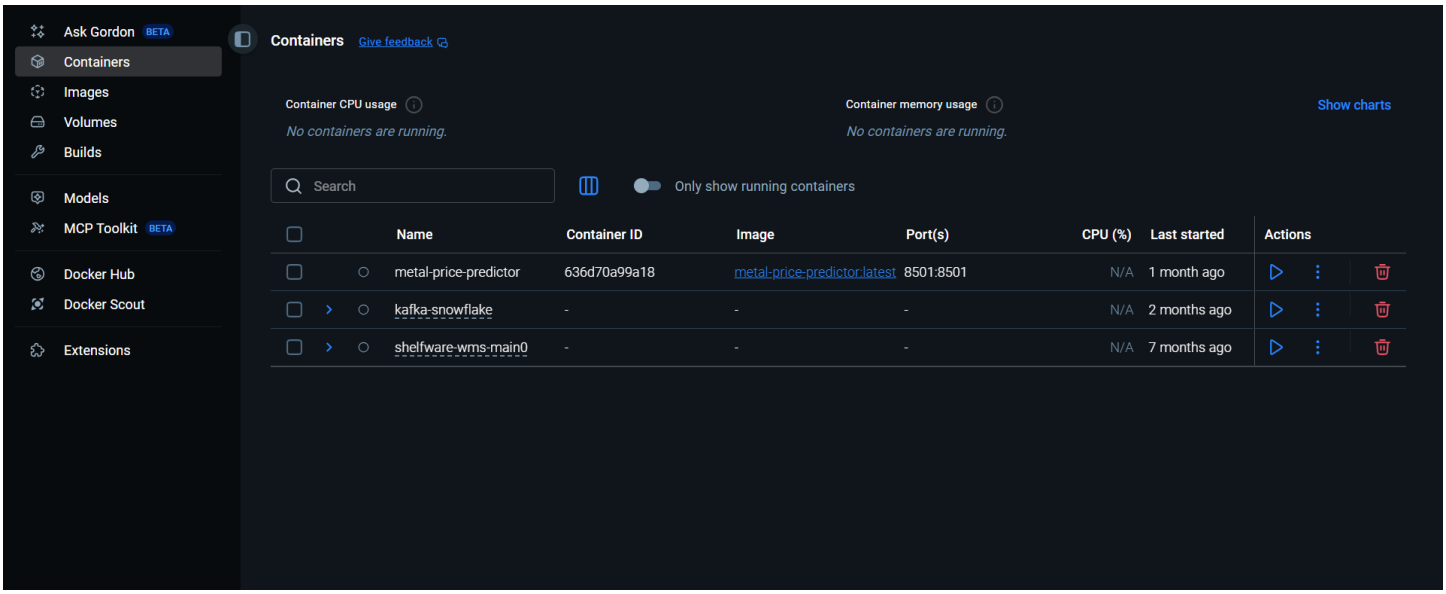


5. Deployment Process (Local)

```
# Step 1: Clone the repository
git clone https://github.com/yourusername/carrier-selection-agent.git
cd carrier-selection-agent

# Step 2: Build and start all containers
docker-compose up --build -d

# Step 3: Verify containers are running
docker ps

# Step 4: Access the application
# React Frontend       -> http://localhost:3000
# FastAPI Backend      -> http://localhost:8000
# Swagger API Docs     -> http://localhost:8000/docs

# Step 5: Check logs
docker logs carrier-backend
docker logs carrier-frontend

# Step 6: Stop containers
docker-compose down
```

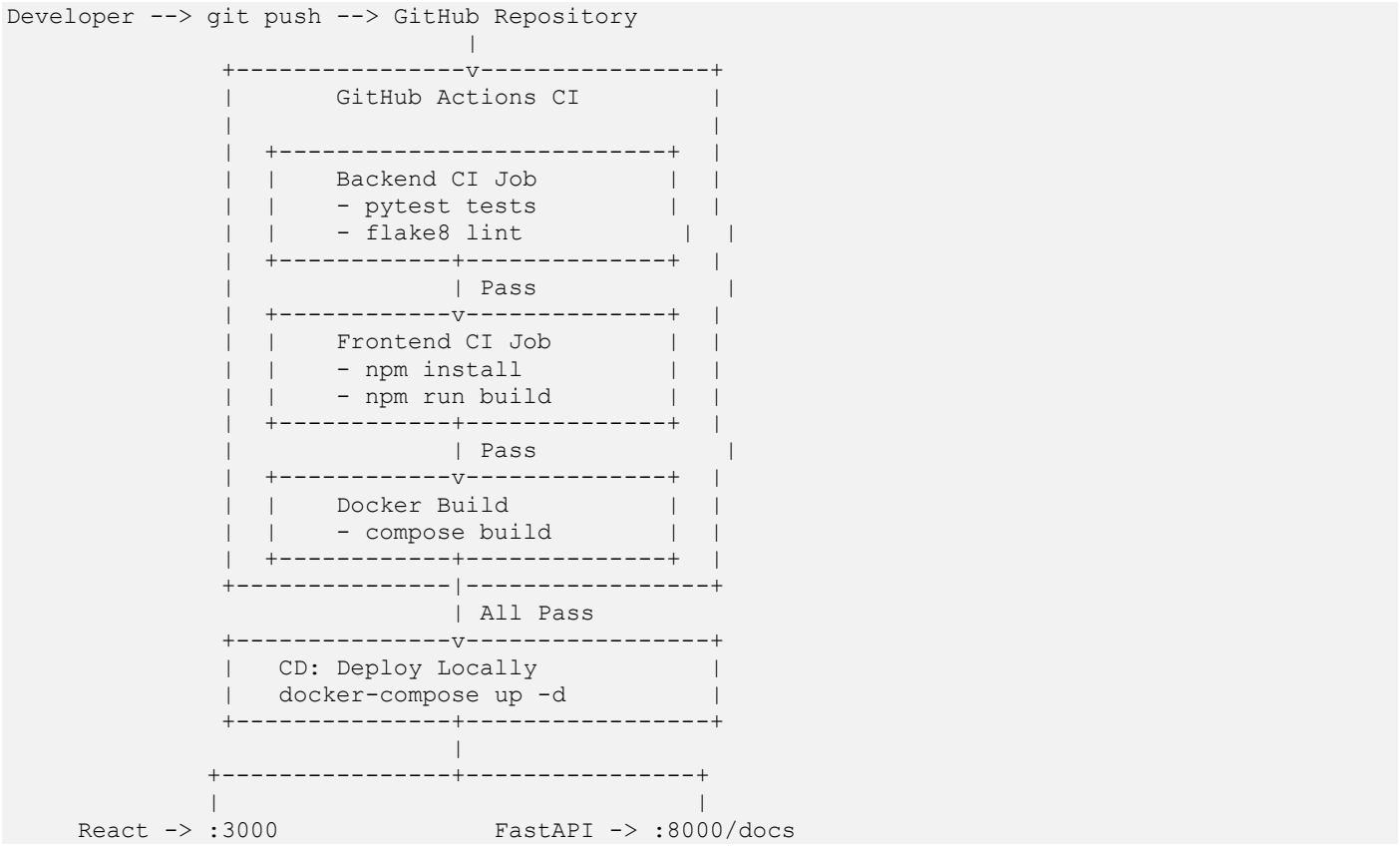
Expected docker ps Output

```
CONTAINER ID   IMAGE               PORTS                     NAMES
b1c2d3e4f5a6   carrier-frontend    0.0.0.0:3000->80/tcp      carrier-frontend
a6b7c8d9e0f1   carrier-backend     0.0.0.0:8000->8000/tcp    carrier-backend
```



6. CI/CD Workflow Explanation

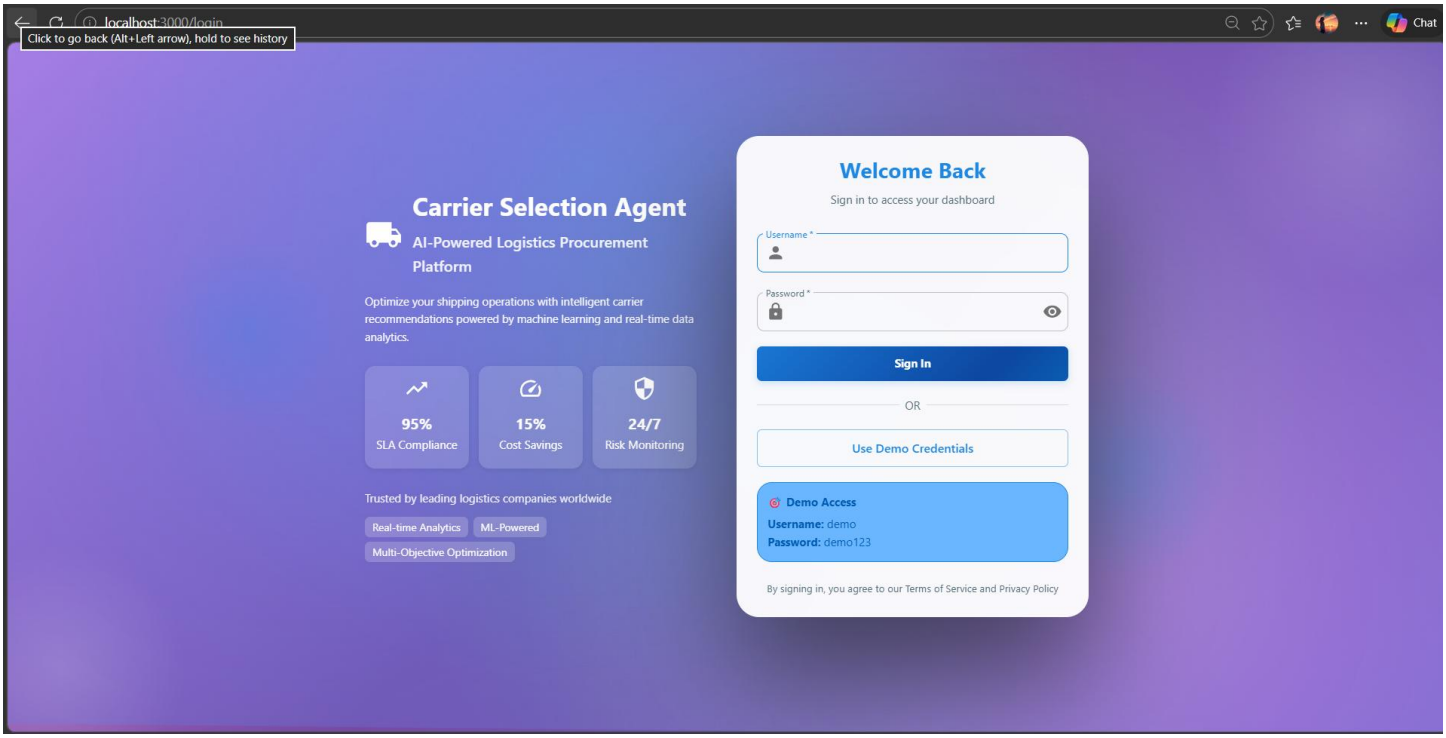
The CI/CD workflow connects code changes directly to deployment, reducing manual effort and improving reliability. Below is the complete flow:

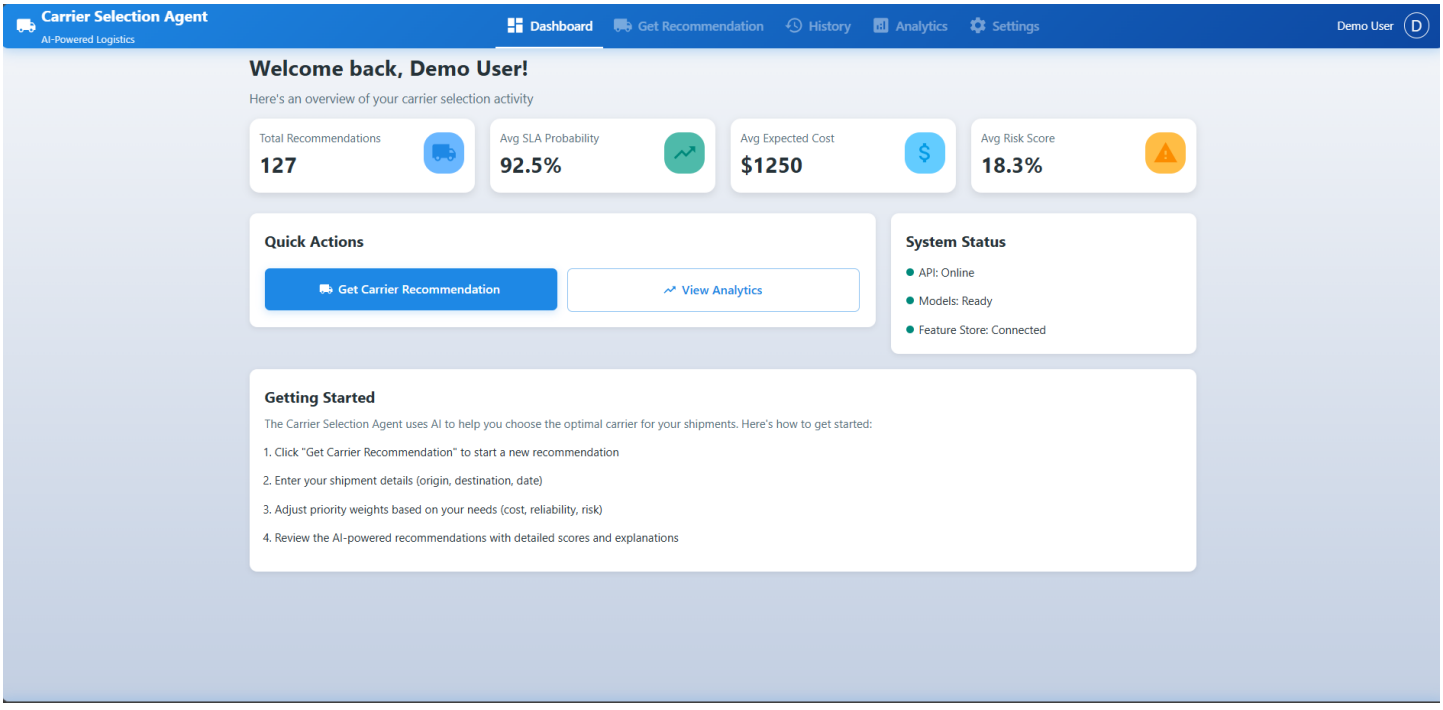


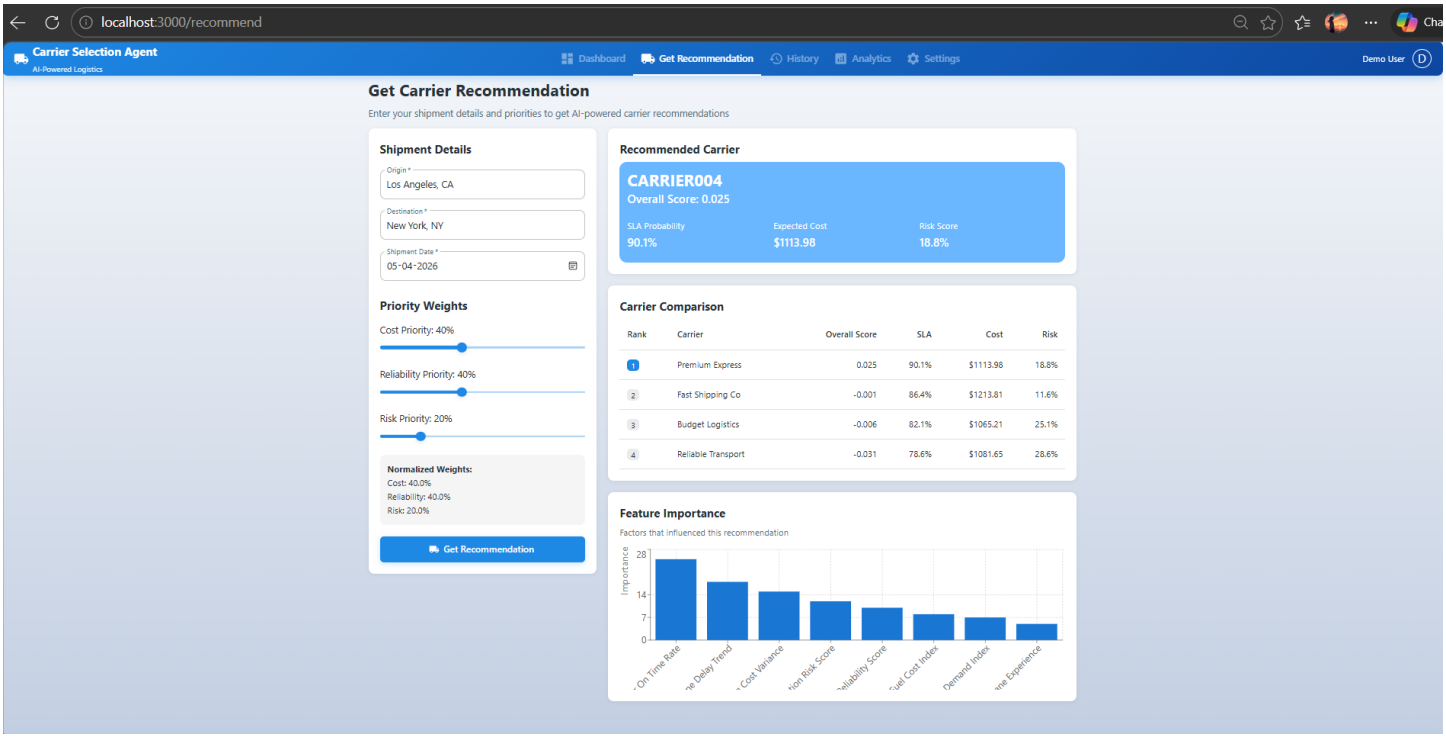
Stage	Tool	Purpose
Source Control	Git + GitHub	Track and manage code
CI Trigger	GitHub Actions	Auto-run on push/PR
Backend Testing	pytest + httpx	Validate API logic
Frontend Build	Node.js + npm	Compile React app
Linting	flake8	Enforce code quality
Containerization	Docker Compose	Build both services
Deployment	Docker Compose up	Launch locally

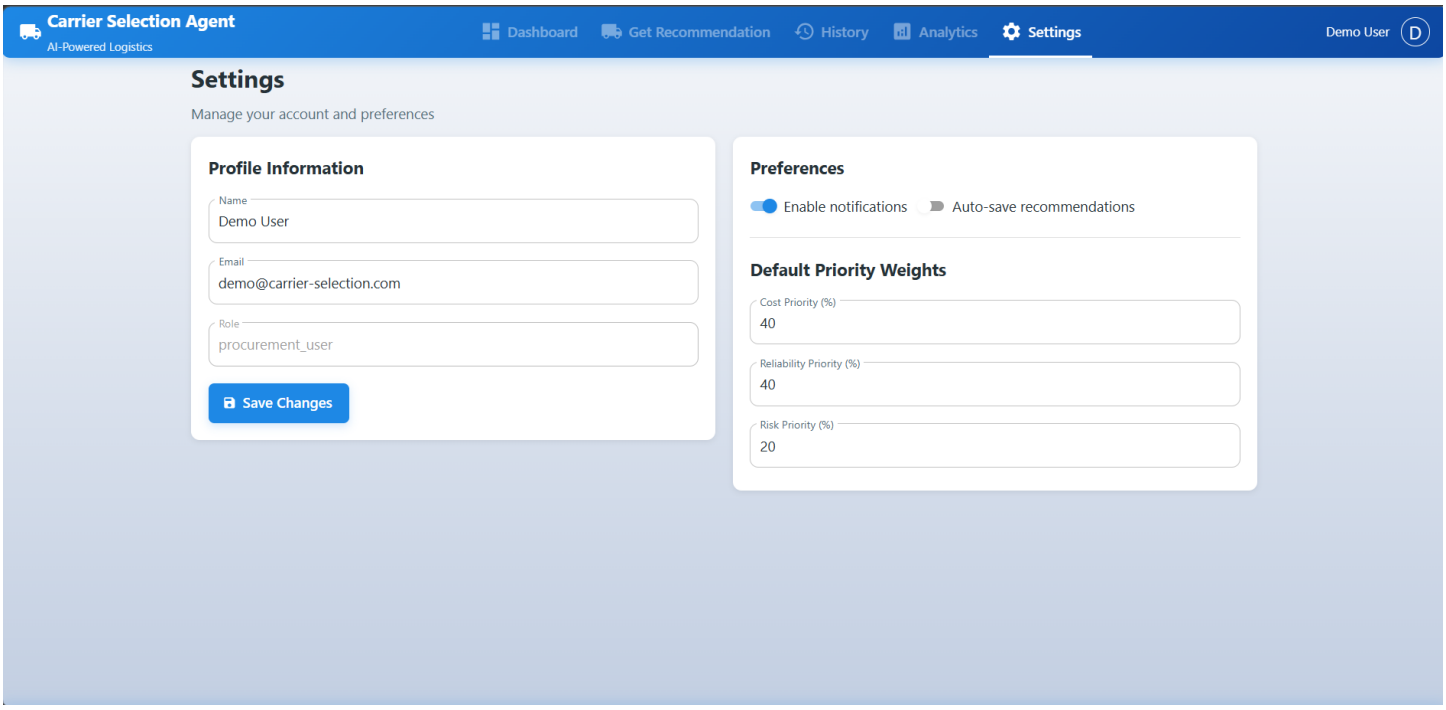
7. Screenshots of Each Stage

The following table describes the screenshots to be captured and attached at each stage of the DevOps lifecycle:






localhost:3000/recommend
Carrier Selection Agent
AI-Powered Logistics
Dashboard
Get Recommendation
History
Analytics
Settings
Demo User
Get Carrier Recommendation
Enter your shipment details and priorities to get AI-powered carrier recommendations
Shipment Details
Origin *
Los Angeles, CA
Destination *
New York, NY
Shipment Date *
05-04-2026
Priority Weights
Cost Priority: 40%
Reliability Priority: 40%
Risk Priority: 20%
Normalized Weights:
Cost: 40.0%
Reliability: 40.0%
Risk: 20.0%
Get Recommendation
Recommended Carrier
CARRIER004
Overall Score: 0.025
SLA Probability
90.1%
Expected Cost
$1113.98
Risk Score
18.8%
Carrier Comparison
Rank
Carrier
Overall Score
SLA
Cost
Risk
1
Premium Express
0.025
90.1%
$1113.98
18.8%
2
Fast Shipping Co
-0.001
86.4%
$1213.81
11.6%
3
Budget Logistics
-0.006
82.1%
$1065.21
25.1%
4
Reliable Transport
-0.031
78.6%
$1081.65
28.6%
Feature Importance
Factors that influenced this recommendation
Importance
28
14
7
0


Carrier Selection Agent
AI-Powered Logistics
Dashboard
Get Recommendation
History
Analytics
Settings
Demo User
Settings
Manage your account and preferences
Profile Information
Name
Demo User
Email
demo@carrier-selection.com
Role
procurement_user
Save Changes
Preferences
Enable notifications
Auto-save recommendations
Default Priority Weights
Cost Priority (%)
40
Reliability Priority (%)
40
Risk Priority (%)
20

#	Stage	What the Screenshot Shows
1	Code – Backend	VS Code showing main.py and carrier_agent.py with FastAPI routes
2	Code – Frontend	VS Code showing App.jsx with React form and Axios API call
3	GitHub Repository	GitHub repo page with folder structure, commits, and branches
4	GitHub Actions – CI	Actions tab showing 3 jobs (Backend CI, Frontend CI, Docker Build) with green checkmarks
5	Docker Build	Terminal output of docker-compose up --build completing successfully
6	Docker PS	Terminal showing both containers running on ports 3000 and 8000
7	React Frontend	Browser at http://localhost:3000 showing the carrier selection form
8	API Result	React UI displaying the recommended carrier card after form submission
9	FastAPI Swagger Docs	Browser at http://localhost:8000/docs showing auto-generated API documentation
10	Test Results	Terminal showing all pytest test cases passing

8. Challenges Faced and Solutions

#	Challenge	Solution
1	CORS error – React frontend blocked by FastAPI backend	Added CORSMiddleware in FastAPI with allow_origins=["*"]
2	React build failed in Docker – Missing environment variables	Used multi-stage Docker build and ensured all env vars were set at build time
3	Port conflict – Port 3000 already occupied locally	Changed Docker Compose host port mapping to 3001:80
4	GitHub Actions Docker build slow – Full image rebuilt on every push	Added Docker layer caching using actions/cache in the workflow
5	FastAPI not reachable from React in Docker – Services could not communicate	Used Docker Compose service names as hostnames for inter-container communication
6	Pydantic v2 breaking changes – BaseModel behavior changed	Updated model definitions to use Pydantic v2 syntax and pinned version in requirements.txt

9. Conclusion

This assignment successfully demonstrated a complete DevOps lifecycle using a practical AI-powered application — the Carrier Selection Agent for Logistics.

- FastAPI provided a high-performance, auto-documented REST backend with built-in Swagger UI, making API testing and integration straightforward.
- React delivered a clean, interactive frontend that communicated seamlessly with the backend via Axios.
- Docker and Docker Compose containerized both services, ensuring consistent and reproducible environments across development and deployment.
- GitHub Actions automated the entire CI pipeline — testing, linting, building, and validating both frontend and backend on every code push.
- Git branching enforced structured development with feature branches, clean commits, and controlled merges.

The project reinforced that DevOps is not just a set of tools — it is a mindset of automation, collaboration, and continuous delivery. The Carrier Selection Agent, though a focused application, showcased how modern AI-powered services are built, tested, and deployed following industry-standard DevOps practices.

The integration of FastAPI and React within a Dockerized environment, monitored by automated GitHub Actions workflows, reflects the real-world standards adopted by modern software engineering teams. This assignment provided hands-on experience with every layer of the DevOps lifecycle — from writing code and managing versions, to containerizing services and automating builds and deployments.